UNIT II

REQUIREMENTS ANALYSIS AND SPECIFICATION

Requirement analysis and specification – Requirements gathering and analysis – Software Requirement Specification – Formal system specification – Finite State Machines – Petrinets – Object modelling using UML – Use case Model – Class diagrams – Interaction diagrams – Activity diagrams – State chart diagrams – Functional modelling – Data Flow Diagram- CASE TOOLS.

2.1 Software requirements

Requirements engineering is the process of gathering, analyzing, documenting, validating, and managing requirements. Requirements analysis is a software engineering task that bridges the gap between system level requirements engineering and software design.

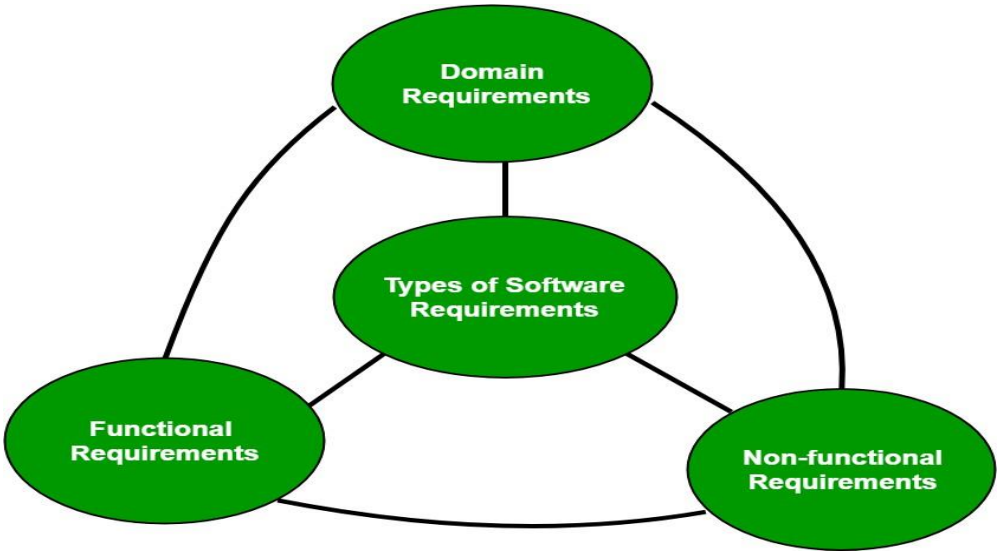
The software requirements are description of features and functionalities of the target system. Requirements convey the expectations of users from the software product. The requirements can be obvious or hidden, known or unknown, expected or unexpected from client's point of view.

Main types of software requirement can be of 3 types

Functional requirements

Non-functional requirements

Domain requirements



Common classifications of software requirements can be:

1. **User requirements:** These requirements describe what the end-user wants from the software system. User requirements are usually expressed in natural language and are typically gathered through interviews, surveys, or user feedback.
2. **System requirements:** These requirements specify the technical characteristics of the software system, such as its architecture, hardware requirements, software components, and interfaces. System requirements are typically expressed in technical terms and are often used as a basis for system design.
3. **Business requirements**: These requirements describe the business goals and objectives that the software system is expected to achieve. Business requirements are usually expressed in terms of revenue, market share, customer satisfaction, or other business metrics.
4. **Regulatory requirements:** These requirements specify the legal or regulatory standards that the software system must meet. Regulatory requirements may include data privacy, security, accessibility, or other legal compliance requirements.
5. **Interface requirements**: These requirements specify the interactions between the software system and external systems or components, such as databases, web services, or other software applications.
6. **Design requirements:** These requirements describe the technical design of the software system. They include information about the software architecture, data structures, algorithms, and other technical aspects of the software.

2.1.1 Functional Requirements

It defines describes functionality or system services. Functional requirements are the behavior or functions that the system must support. These are the attributes that characterize what the software does to fulfill the needs of the customer. These can be business rules, administrative tasks, transactions, cancellations, authentication, authorization, external interfaces, legal or regulatory requirements, audit tracking, certification, reporting requirements, and historical data.

There are many ways of expressing functional requirements e.g., natural language, a structured or formatted language with no rigorous syntax and formal specification language with proper syntax. These are represented or stated in the form of input to be given to the system, the operation

performed and the output expected. They are basically the requirements stated by the user which one can see directly in the final product, unlike the non-functional requirements.

2.1.2 Nonfunctional Requirements

Nonfunctional requirements specify how a system must behave. These are qualities, standards, constraints upon the systems services that are specified with respect to a product, organization, and external environment. Nonfunctional requirements are related to functional requirements, i.e., how efficiently, by how much volume, how fast, at what quality, how safely, etc., a function is performed by a particular system. The examples of nonfunctional requirements are reliability, maintainability, performance, usability, security, scalability, capacity, availability, recoverability, serviceability, manageability, integrity, and interoperability.

Nonfunctional Requirements for ETransQ

- Availability: ETransQ is online information, which will be accessible 24 x 7 hours.
- Recoverability: The system will maintain backup of dynamically changing requirement and any loss of data.
- Extensibility: The system is able to adopt new added features by extending the base classes.
- Maintainability: The system is easy to maintain and allows adding new information and removing old information.
- Privacy: Users have full authority to the extent they want their information to be displayed to other users and can also block other users.
- Security: It allows only authorized users-transporters, agents or suppliers to access information resources.
- Compatibility: It provides backward and forward compatibility i.e., the system is accessible from older and newer machines.
- Usability: ETransQ is very user friendly.
- Stability: ETransQ has an optimizing code to reduce load on the application server and database server.
- Reusability: The system is inspired by Object orientation methodology, thus enhancing the reusability of the system.

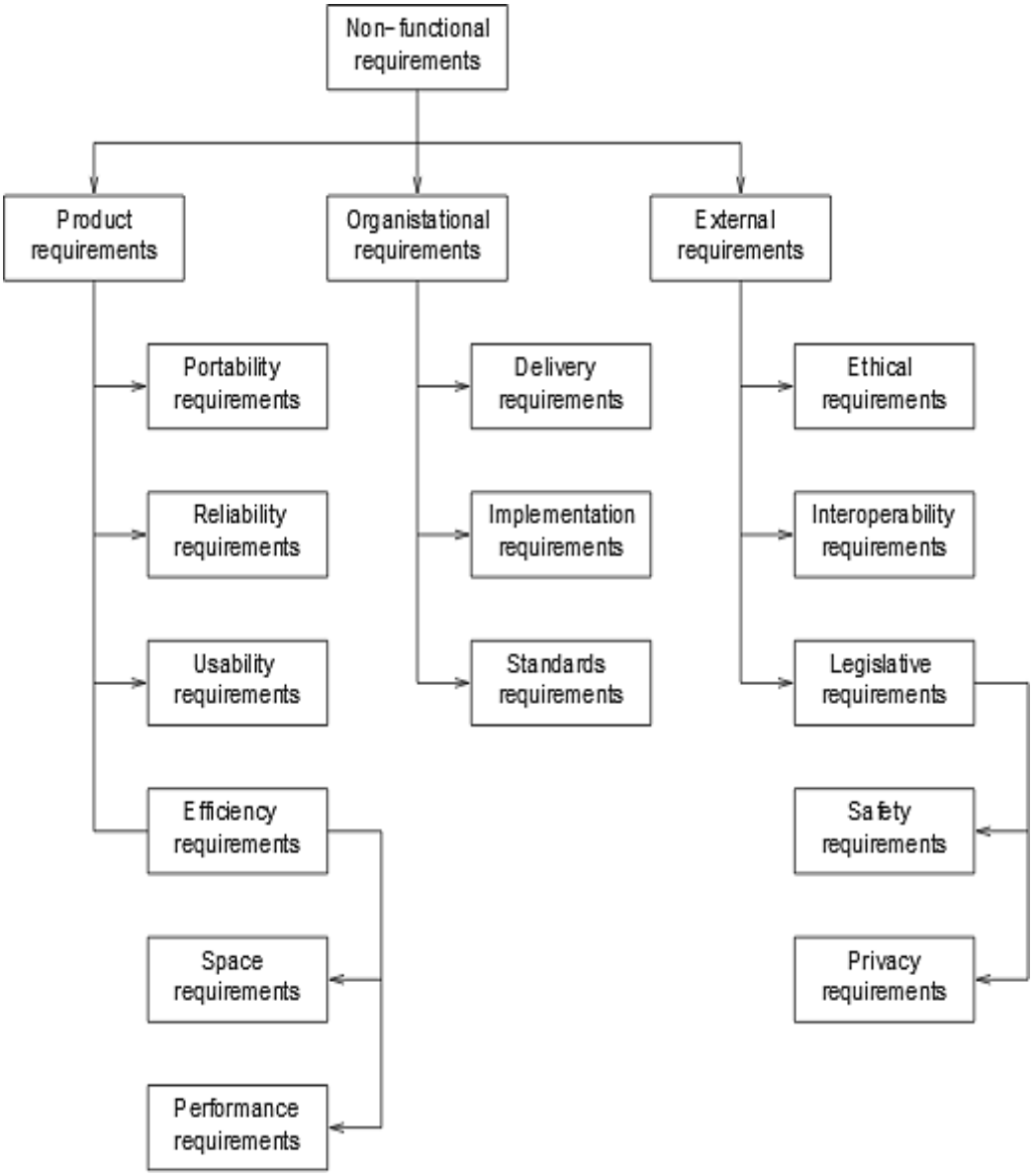


Figure 2.1: Non-functional Requirements

- Robustness: The system being developed consists of validations and checks which will help prevent invalid inputs for smooth functioning of the system.

Domain requirements: Domain requirements are the requirements which are characteristic of a particular category or domain of projects. Domain requirements can be functional or nonfunctional.

Functional Requirements	Non Functional Requirements
A functional requirement defines a system or its component.	A non-functional requirement defines the quality attribute of a software system.
It specifies "What should the software system do?"	It places constraints on "How should the software system fulfill the functional requirements?"
Functional requirement is specified by User.	Non-functional requirement is specified by technical peoples e.g. Architect, Technical leaders and software developers.
It is mandatory.	It is not mandatory.
It is captured in use case.	It is captured as a quality attribute.
Defined at a component level.	Applied to a system as a whole.
Helps you verify the functionality of the software.	Helps you to verify the performance of the software.
Functional Testing like System, Integration, End to End, API testing, etc are done.	Non-Functional Testing like Performance, Stress, Usability, Security testing, etc are done.
Usually easy to define.	Usually more difficult to define.

2.2 Requirement Engineering Process

The process to gather the software requirements from client, analyze and document them is known as requirement engineering. The goal of requirement engineering is to develop and maintain sophisticated and descriptive 'System Requirements Specification' document.

Requirement engineering is the key phase in software development which decides what to build and it provides the outline of the quality of the final product. Requirement engineering is the disciplined, process-oriented approach to the development and management of requirements. Requirement Engineering Process is a five step process, which includes,

- Feasibility Study
- Requirement Gathering/ Requirement Elicitation and analysis
- Software Requirement Specification
- Software Requirement Validation
- Requirements Management

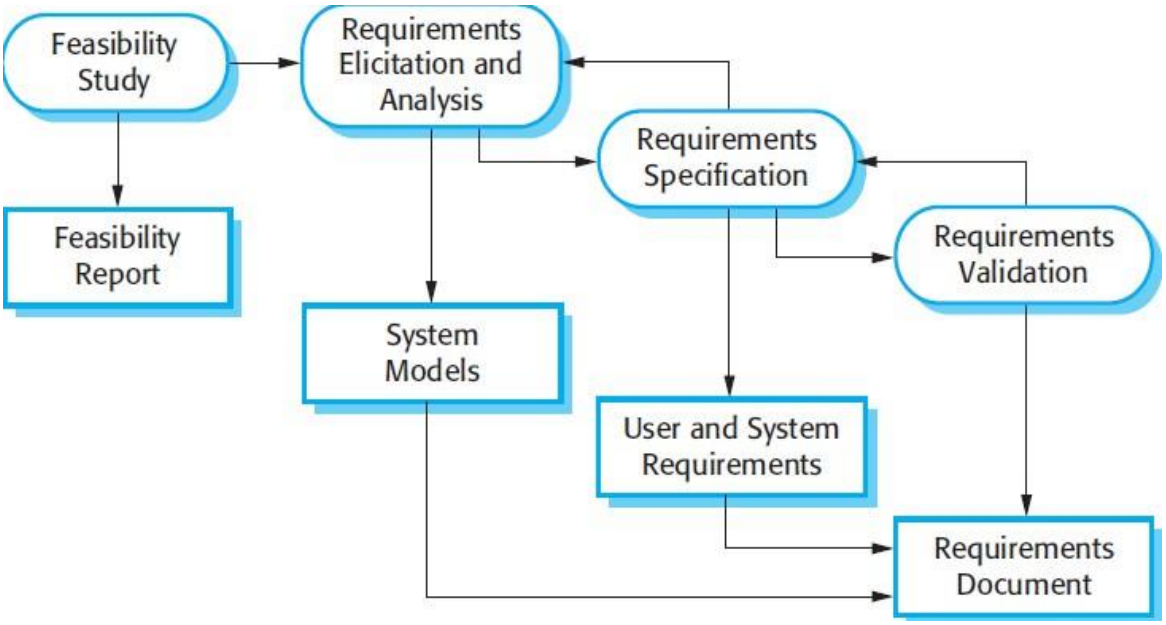


Figure 2.2: Requirement Engineering Process

2.2.1 Feasibility Study

The main aim of a feasibility study is to create reasons for the development of the software that the users accept, that is flexible enough and open to changes, and abide by the standards chosen for software development and maintenance.

There are several types of feasibility. They are:

Technical Feasibility: The current technologies are evaluated using technical feasibility, and they are necessary to achieve the requirements of the customer within the given time and budget.

Operational Feasibility: The assessment of the range for software in which the required software performs a series of levels to solve the problems in business and the customer's requirements.

Economic Feasibility: If the required software can generate profits in the area of finance is decided by economic feasibility.

2.2.2 Requirement Elicitation / Requirement Gathering

If the feasibility report is positive towards undertaking the project, next phase starts with gathering requirements from the user. Requirements Elicitation is the process to find out the requirements for an intended software system by communicating with client, end users, system users and others who have a stake in the software system development.

Requirement elicitation aims to gather requirements from different perspectives to understand the customer needs in the real scenario.

Analysts and engineers communicate with the client and end-users to know their ideas on what the software should provide and which features they want the software to include.

Requirement Elicitation Techniques (Requirement gathering)

There are various ways to discover requirements. The various techniques are,

Interviews

Interviews are strong medium to collect requirements. Organization may conduct several types of interviews such as:

- Structured (closed) interviews, where every single information to gather is decided in advance, they follow pattern and matter of discussion firmly.
- Non-structured (open) interviews, where information to gather is not decided in advance, more flexible and less biased.
- Oral interviews
- Written interviews
- One-to-one interviews which are held between two persons across the table.
- Group interviews which are held between groups of participants. They help to uncover any missing requirement as numerous people are involved.

Surveys

Organization may conduct surveys among various stakeholders by querying about their expectation and requirements from the upcoming system.

Questionnaires

- A document with pre-defined set of objective questions and respective options is handed over to all stakeholders to answer, which are collected and compiled.
- A shortcoming of this technique is, if an option for some issue is not mentioned in the questionnaire, the issue might be left unattended.

Task analysis

Team of engineers and developers may analyze the operation for which the new system is required. If the client already has some software to perform certain operation, it is studied and requirements of proposed system are collected.

Domain Analysis

Every software falls into some domain category. The expert people in the domain can be a great help to analyze general and specific requirements.

Brainstorming

An informal debate is held among various stakeholders and all their inputs are recorded for further requirements analysis.

Prototyping

Prototyping is building user interface without adding detail functionality for user to interpret the features of intended software product. It helps giving better idea of requirements. If there is no software installed at client's end for developer's reference and the client is not aware of its own requirements, the developer creates a prototype based on initially mentioned requirements. The prototype is shown to the client and the feedback is noted. The client feedback serves as an input for requirement gathering.

Observation

Team of experts visits the client's organization or workplace. They observe the actual working of the existing installed systems. They observe the workflow at client's end and how execution problems are dealt. The team itself draws some conclusions which aid to form requirements expected from the software.

2.2.3 Software Requirement Specification

SRS is a document created by system analyst after the requirements are collected from various stakeholders. SRS defines how the intended software will interact with hardware, external

interfaces, speed of operation, response time of system, portability of software across various platforms, maintainability, speed of recovery after crashing, Security, Quality, Limitations etc.

The requirements received from client are written in natural language. It is the responsibility of system analyst to document the requirements in technical language so that they can be comprehended and useful by the software development team.

SRS should come up with following features:

- User Requirements are expressed in natural language.
- Technical requirements are expressed in structured language, which is used inside the organization.
- Design description should be written in Pseudo code.
- Format of Forms and GUI screen prints.
- Conditional and mathematical notations for DFDs etc.

2.2.4 Software Requirement Validation

After requirement specifications are developed, the requirements mentioned in this document are validated. User might ask for illegal, impractical solution or experts may interpret the requirements incorrectly. This results in huge increase in cost if not nipped in the bud. Requirements can be checked against following conditions -

- If they can be practically implemented
- If they are valid and as per functionality and domain of software
- If there are any ambiguities
- If they are complete
- If they can be demonstrated

2.2.5 Requirements management

Requirement management is the process of analyzing, documenting, tracking, prioritizing and agreeing on the requirement and controlling the communication to relevant stakeholders. This stage takes care of the changing nature of requirements.

It should be ensured that the SRS is as modifiable as possible so as to incorporate changes in requirements specified by the end users at later stages too. Being able to modify the software as per requirements in a systematic and controlled manner is an extremely important part of the requirements engineering process.

2.3 Software Requirement Specification

A software requirements specification (SRS) is a document explaining how and what the software/system will do. It defines the features and functionality that the product requires to satisfy all stakeholders' (business, users) needs.

A standard SRS includes:

- A goal/purpose
- A summary of the whole process
- Specific Requirements

Requirements document structure

The IEEE standard suggests the following standard:

- Introduction
- Glossary
- User requirements definition
- System architecture
- System requirements specification
- System models
- System evolution
- Appendices and
- Index

SRS Contains the following

Introduction

- **Purpose of this Document –** At first, main aim of why this document is necessary and what's purpose of document is explained and described.
- **Scope of this document –** In this, overall working and main objective of document and what value it will provide to customer is described and explained. It also includes a description of development cost and time required.
- **Overview –** In this, description of product is explained. It's simply summary or overall review of product.

General description